3D Orbital Mechanics & Numerical Integration

Grade 11 Physics Project: Groups 3 & 8

St. John Baptist De La Salle

March 2026

The Calculus of Motion

In classical mechanics, satellite movement is defined by derivatives:

- **Velocity ($\vec{v}$):** $\vec{v} = \frac{d\vec{r}}{dt}$
- **Acceleration ($\vec{a}$):** $\vec{a} = \frac{d\vec{v}}{dt}$

Newton's Law of Universal Gravitation provides the force:

$$\vec{a} = -\frac{GM}{r^2}\hat{r}$$

The Importance of the Time Step (dt)

Computers cannot process continuous time. We use a finite dt:

Linear Approximation

$$\Delta\vec{r} \approx \vec{v} \cdot dt \implies \vec{r}_{new} = \vec{r}_{old} + \vec{v} \cdot dt$$

A small dt (e.g., 2.0s) ensures the satellite follows the curve of the orbit accurately without "overshooting."

Euler's Method: The Core Engine

Our code uses Euler integration to update the satellite's state:

1. Calculate $\vec{a}$ based on current position $\vec{r}$.
2. Update Velocity: $\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n dt$
3. Update Position: $\vec{r}_{n+1} = \vec{r}_n + \vec{v}_{n+1} dt$

Implementation in VPython

Dydg by mamejemal355e 2026/03/18 20:05:58

Run this program Share or export this program Download

Signed in as mamejemal355e(Sign out)

Help

```python
Web VPython 3.2
from vpython import *
"""
==============================================================================
ST JOHN BAPTIST DE LA SALLE - GRADE 11 PHYSICS PROJECT
Task 3: 3D Orbital Mechanics & Satellites (Groups 3 & 8)
Method: Numerical Integration via Euler's Method
==============================================================================
"""

# ==============================================================================
# 1. PHYSICAL CONSTANTS
# ==============================================================================
G = 6.67430e-11         # Gravitational Constant (N m^2 / kg^2)
M_EARTH = 5.972e24      # Mass of Central Body (kg)
R_EARTH = 6378137.0     # Radius of Central Body (m)

# ==============================================================================
# 2. 3D VISUALIZATION SETUP
# ==============================================================================
scene = canvas(title="<b>Task 3: Orbital Mechanics (Euler Integration)</b>",
               width=1200, height=650, background=color.black)
scene.ambient = vec(0.4, 0.4, 0.4)
scene.autoscale = False # Prevents camera from jumping around

# Force the initial camera position
scene.camera.pos = vec(0, 0, R_EARTH * 4.5)
scene.camera.axis = vec(0, 0, -R_EARTH * 4.5)

# ------------------------------------------------------------------------------
# COSMETICS: STARFIELD & EARTH (USING EMISSIVE SPHERES)
# ------------------------------------------------------------------------------
for _ in range(300):
    sphere(pos=vec(random()-0.5, random()-0.5, random()-0.5) * 4e8,
           radius=random()*8e5, color=color.white, emissive=True)

earth = sphere(pos=vec(0, 0, 0), radius=R_EARTH, texture=textures.earth)

# Custom 3D Satellite
sat_body = cylinder(pos=vec(-R_EARTH/20, 0, 0), axis=vec(R_EARTH/10, 0, 0), radius=R_EARTH/30, color=vec(0.7, 0.7,
sat_panels = box(pos=vec(0, 0, 0), size=vec(R_EARTH/15, R_EARTH/80, R_EARTH/5), color=color.blue)
sat = compound([sat_body, sat_panels], pos=vec(R_EARTH + 5000000, 0, 0),
               make_trail=True, trail_type="curve", retain=3000)

v_arrow = arrow(color=color.green, shaftwidth=R_EARTH/25)
f_arrow = arrow(color=color.red, shaftwidth=R_EARTH/25)

# THE FIX: Bumped the Y-position from 20 to 120 so the text doesn't fall off the screen!
hud = label(pixel_pos=True, pos=vec(20, 120, 0), text="",
            box=False, color=color.white, align='left', height=16, font='monospace')

# ------------------------------------------------------------------------------
# DUAL TELEMETRY GRAPHS
# ------------------------------------------------------------------------------
scene.append_to_caption('\n')
energy_graph = graph(title="<b>Specific Orbital Energy (Euler Tracking)</b>",
                     xtitle="Time (s)", ytitle="Energy (J/kg)", width=1200, height=220)
curve_ke = gcurve(graph=energy_graph, color=color.green, label="Kinetic Energy")
curve_pe = gcurve(graph=energy_graph, color=color.red, label="Potential Energy")
curve_te = gcurve(graph=energy_graph, color=color.yellow, label="Total Energy")

scene.append_to_caption('\n')
orbit_graph = graph(title="<b>Altitude & Velocity vs Time</b>",
                    xtitle="Time (s)", ytitle="Magnitude", width=1200, height=220)
curve_alt = gcurve(graph=orbit_graph, color=color.cyan, label="Altitude (m)")
curve_vel = gcurve(graph=orbit_graph, color=color.magenta, label="Velocity (m/s) x1000")

# ==============================================================================
# 3. TRAJECTORY INITIALIZATION & UI LOGIC
# ==============================================================================
sim_state = {'t': 0, 'dt': 2.0, 'warp': 200, 'paused': False, 'camera_mode': 'Free', 'traj': ''}

def reset_simulation():
    sim_state['t'] = 0
    sat.pos = vec(R_EARTH + 5000000, 0, 0)
    sat.clear_trail()
    curve_ke.data = []
    curve_pe.data = []
    curve_te.data = []
    curve_alt.data = []
```

Dydg by mamejemal355e 2026/03/18 20:05:58

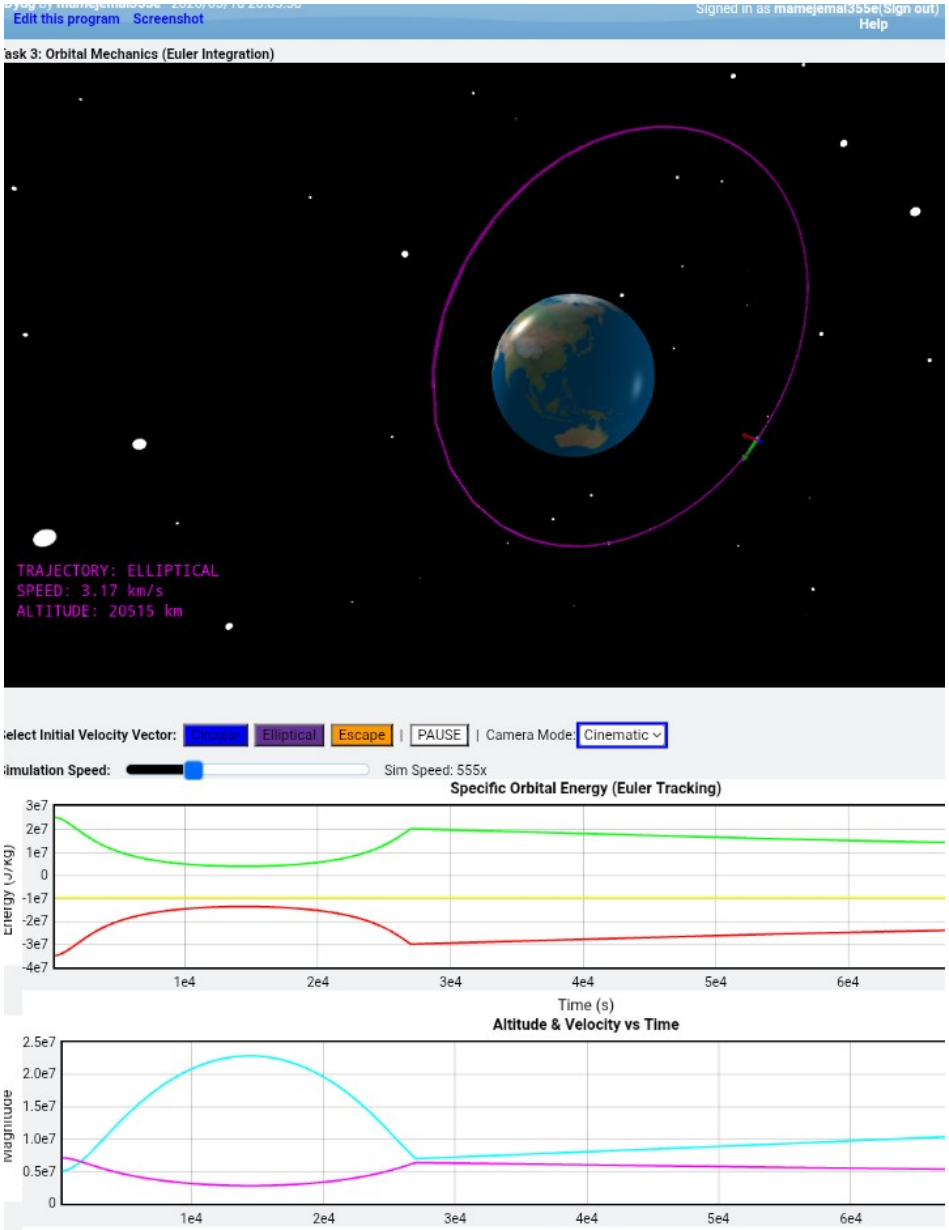
Run this program Share or export this program Download

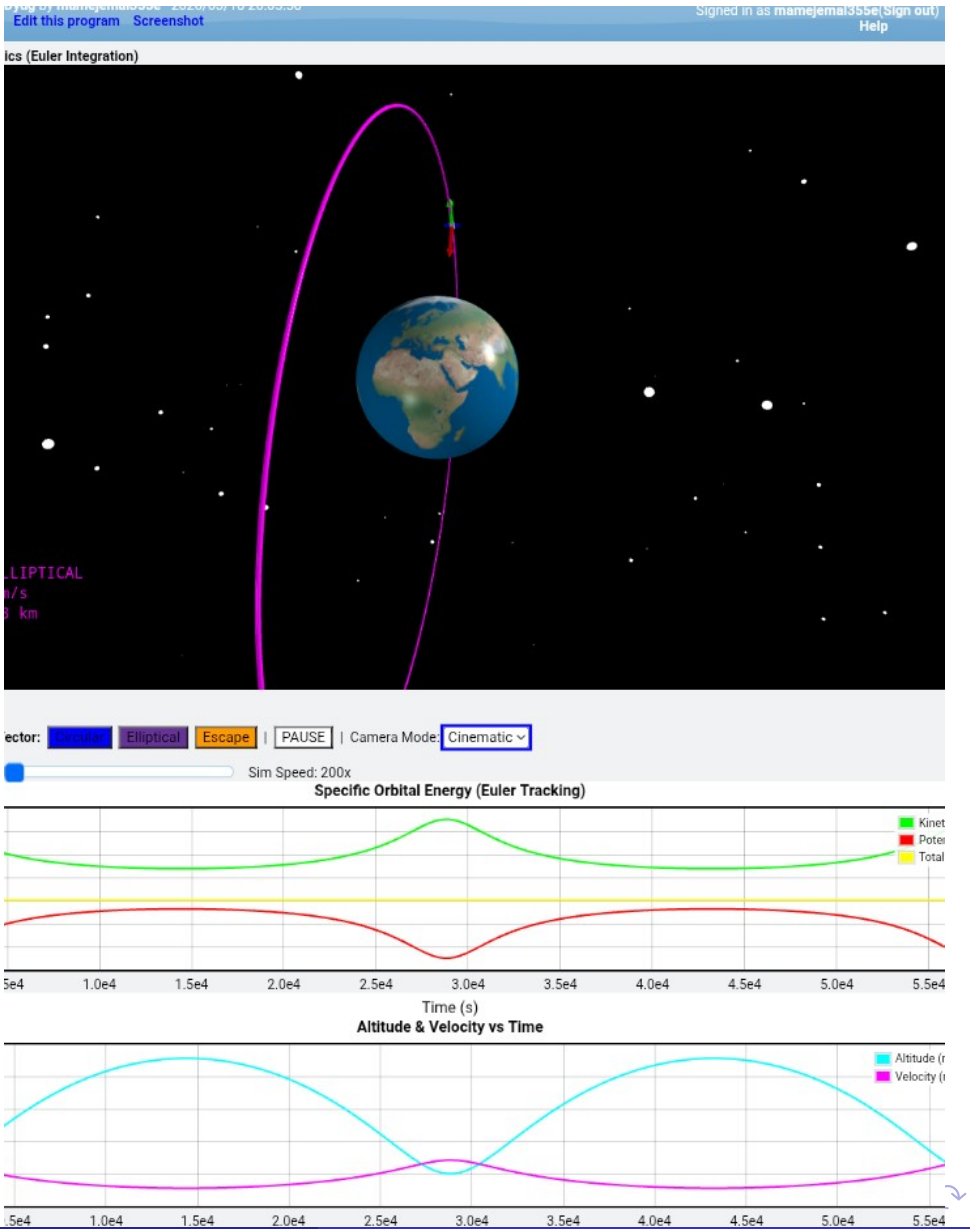
Signed in as mamejemal355e(Sign out)

Help

```python
def inject_escape(b):
    reset_simulation()
    r_mag = mag(sat.pos)
    v_mag = sqrt(2 * G * M_EARTH / r_mag) * 1.05
    sat.vel = vec(0, v_mag, 0)
    sat.trail_color = color.orange
    hud.color = color.orange
    sim_state['traj'] = "ESCAPE (HYPERBOLIC)"

def toggle_pause(b):
    sim_state['paused'] = not sim_state['paused']
    b.text = "RESUME" if sim_state['paused'] else "PAUSE"

def set_camera(m):
    sim_state['camera_mode'] = m.selected
    if m.selected == 'Free':
        scene.camera.pos = vec(0, 0, R_EARTH * 4.5)
        scene.camera.axis = vec(0, 0, -R_EARTH * 4.5)

def set_speed(s):
    sim_state['warp'] = s.value
    speed_txt.text = " Sim Speed: " + str(int(s.value)) + "x"

# UI Controls
scene.append_to_caption('<b>Select Initial Velocity Vector:</b>  ')
button(text="Circular", bind=inject_circular, background=color.blue)
scene.append_to_caption('  ')
button(text="Elliptical", bind=inject_elliptical, background=color.purple)
scene.append_to_caption('  ')
button(text="Escape", bind=inject_escape, background=color.orange)
scene.append_to_caption('  |  ')
button(text="PAUSE", bind=toggle_pause)
scene.append_to_caption('  |  Camera Mode: ')
menu(choices=['Free', 'Follow', 'Cinematic'], bind=set_camera)
scene.append_to_caption('\n\n<b>Simulation Speed:</b> ')
slider(bind=set_speed, min=10, max=2000, value=200, length=300)
speed_txt = wtext(text=" Sim Speed: 200x")

inject_elliptical(None)

# ==============================================================================
# 4. MAIN PHYSICS LOOP (EULER'S METHOD)
# ==============================================================================
while True:
    rate(120)

    if not sim_state['paused']:
        steps = int(sim_state['warp'] / sim_state['dt'])

        for _ in range(steps):
            # STEP 1: GRAVITATIONAL FORCE
            r_vec = sat.pos - earth.pos
            r_mag = mag(r_vec)
            r_hat = norm(r_vec)
            a_vec = - (G * M_EARTH / r_mag**2) * r_hat

            # STEP 2: EULER'S METHOD
            sat.vel = sat.vel + a_vec * sim_state['dt']
            sat.pos = sat.pos + sat.vel * sim_state['dt']
            sim_state['t'] += sim_state['dt']

        # ------------------------------------------------------------
        # VISUALS & TELEMETRY
        # ------------------------------------------------------------
        earth.rotate(angle=7.29e-5 * sim_state['warp'], axis=vec(0, 1, 0))
        sat.axis = norm(sat.vel) * (R_EARTH/5)

        v_arrow.pos = sat.pos
        v_arrow.axis = norm(sat.vel) * 3e6
        f_arrow.pos = sat.pos
        f_arrow.axis = norm(a_vec) * 3e6

        if sim_state['camera_mode'] == 'Follow':
            scene.camera.pos = sat.pos + norm(sat.pos) * 3e6 + vec(0, 1.5e6, 0)
            scene.camera.axis = sat.pos - scene.camera.pos
        elif sim_state['camera_mode'] == 'Cinematic':
            angle = sim_state['t'] * 5e-5
            scene.camera.pos = vec(4e7 * cos(angle), 1.5e7, 4e7 * sin(angle))
            scene.camera.axis = earth.pos - scene.camera.pos

        # Calculations for HUD and Graphs
```

Simulation Results





Conservation of Orbital Energy

We track Specific Mechanical Energy (E):

$$E = \frac{1}{2}v^2 - \frac{GM}{r}$$

- At **Perigee**: Velocity is max, Potential Energy is min.
- At **Apogee**: Velocity is min, Potential Energy is max.
- Total Energy remains constant.

Conclusion

- Bridged the gap between "smooth" physics and "discrete" computation.
- Demonstrated how Euler's method simulates complex 3D motion.
- Validated results through visual paths and energy telemetry.

Thank You!